

MVLU COLLEGE
A.I. Practical No.9

- Aim :-
- Implement the A* Search algorithm for solving a pathfinding problem.
 - Implement the Recursive Best-First Search algorithm for the same problem.
 - Compare the performance and effectiveness of both algorithms.

Code:-

```
Practical 2.py - D:\Har\AI\Practical 2.py (3.13.5)
File Edit Format Run Options Window Help
import heapq
import networkx as nx
import matplotlib.pyplot as plt
print("Hariprasad Vishwakarma T127")
# PART 1: A* Algorithm (Lucknow to Varanasi)
graph_astar={
    'Lucknow':{'Kanpur':12,'Ayodhya':14},
    'Kanpur':{'Prayagraj':18,'Ayodhya':20},
    'Ayodhya':{'Azamgarh':16},
    'Prayagraj':{'Azamgarh':12,'Varanasi':15},
    'Azamgarh':{'Varanasi':10},
    'Varanasi':{}
}

h_astar={
    'Lucknow':45,
    'Kanpur':34,
    'Ayodhya':30,
    'Prayagraj':22,
    'Azamgarh':12,
    'Varanasi':0
}

pos_astar={
    'Lucknow':(1,9),
    'Kanpur':(3,7),
    'Ayodhya':(1,7),
    'Prayagraj':(3,4),
    'Azamgarh':(1,2),
    'Varanasi':(3,0)
}

def astar(graph,h,start,goal):
    pq=[(h[start],start,[start],0)]
    visited=set()
    while pq:
        f,node,path,g=heapq.heappop(pq)
        if node in visited:
            continue
        if node==goal:
            return path,g
        visited.add(node)
```

MVLU COLLEGE

A.I. Practical No.9

```
Practical 2.py - D:\Har\AI\Practical 2.py (3.13.5)
File Edit Format Run Options Window Help

    visited.add(node)
    for nxt,cost in graph[node].items():
        if nxt not in visited:
            ng=g+cost
            heapq.heappush(pq, (ng+h[nxt],nxt,path+[nxt],ng))
    return None,float('inf')

path_astar,dist_astar=astar(graph_astar,h_astar,'Lucknow','Varanasi')

print("=== A* Search Algorithm ===")
print("Optimal Path Found:")
print(" -> ".join(path_astar))
print(f"Total Distance = {dist_astar} km\n")

G_astar=nx.DiGraph()
for u in graph_astar:
    for v,w in graph_astar[u].items():
        G_astar.add_edge(u,v,weight=w)

plt.figure(figsize=(10,8))
nx.draw_networkx_nodes(G_astar,pos_astar,node_color='lightblue',node_size=2500)
nx.draw_networkx_edges(G_astar,pos_astar,edge_color='gray',arrows=True)
nx.draw_networkx_edges(G_astar,pos_astar,edgelist=list(zip(path_astar,path_astar[1:])),edge_color='orange',width=4,arrows=True)
nx.draw_networkx_labels(G_astar,pos_astar,labels={n:f"{n}\nh(n)={h_astar[n]}" for n in graph_astar},font_size=9,font_weight='bold')
nx.draw_networkx_edge_labels(G_astar,pos_astar,edge_labels={(u,v):f"{w} km" for u,v,w in G_astar.edges.data('weight')},font_color='red')
plt.title("A* Search Route\nLucknow to Varanasi")
plt.axis('off')
plt.tight_layout()
plt.show()

# PART 2: Recursive Best-First Search (RBFS) (Agra to Gorakhpur)
graph_rbfs={
    'Agra':{'Mathura':8,'Noida':18},
    'Mathura':{'Noida':15,'Kanpur':28},
    'Noida':{'Lucknow':30,'Kanpur':25},
    'Kanpur':{'Gorakhpur':35},
    'Lucknow':{'Gorakhpur':32},
    'Gorakhpur':{}
}

h_rbfs={
    'Agra':55,
    'Mathura':48,
    'Noida':40,
    'Kanpur':30,
    'Lucknow':28,
    'Gorakhpur':0
}

pos_rbfs={
    'Agra':(2,9),
    'Mathura':(1,7),
    'Noida':(3,7),
    'Kanpur':(3,4),
    'Lucknow':(1,3),
    'Gorakhpur':(2,1)
}

def rbfs(graph,h,node,goal,g,limit,path):
    if node==goal:
        return path,g,g
    s=[n,g+c,max(g+c+h[n],g+h[node])] for n,c in graph[node].items()
    if not s:
        return None,float('inf'),float('inf')
    while True:
        s.sort(key=lambda x:x[2])
        best=s[0]
        if best[2]>limit:
            return None,float('inf'),best[2]
        alt=s[1][2] if len(s)>1 else float('inf')
        result,dist,best[2]=rbfs(graph,h,best[0],goal,best[1],min(limit,alt),path+[best[0]])
        if result:
            return result,dist,best[2]

path_rbfs,dist_rbfs,_=rbfs(graph_rbfs,h_rbfs,'Agra','Gorakhpur',0,float('inf'),['Agra'])

print("=== Recursive Best-First Search (RBFS) ===")
print("Optimal Path Found:")
print(" -> ".join(path_rbfs))
print(f"Total Distance = {dist_rbfs} km\n")
```

MVLU COLLEGE

A.I. Practical No.9

```
Practical 2.py - D:\Hari\AI\Practical 2.py (3.13.5)
File Edit Format Run Options Window Help

    'Gorakhpur': (2,1)
}

def rbfs(graph,h,node,goal,g,limit,path):
    if node==goal:
        return path,g,g
    s=[n,g+c,max(g+c+h[n],g+h[node])] for n,c in graph[node].items()
    if not s:
        return None,float('inf'),float('inf')
    while True:
        s.sort(key=lambda x:x[2])
        best=s[0]
        if best[2]>limit:
            return None,float('inf'),best[2]
        alt=s[1][2] if len(s)>1 else float('inf')
        result,dist,best[2]=rbfs(graph,h,best[0],goal,best[1],min(limit,alt),path+[best[0]])
        if result:
            return result,dist,best[2]

path_rbfs,dist_rbfs,_rbfs=(graph_rbfs,h_rbfs,'Agra','Gorakhpur',0,float('inf'),['Agra'])

print("=== Recursive Best-First Search (RBFS) ===")
print("Optimal Path Found:")
print(" -> ".join(path_rbfs))
print(f"Total Distance = {dist_rbfs} km\n")

G_rbfs=nx.DiGraph()
for u in graph_rbfs:
    for v,w in graph_rbfs[u].items():
        G_rbfs.add_edge(u,v,weight=w)

plt.figure(figsize=(10,8))
nx.draw_networkx_nodes(G_rbfs,pos_rbfs,node_color='lightblue',node_size=2500)
nx.draw_networkx_edges(G_rbfs,pos_rbfs,edge_color='gray',arrows=True)
nx.draw_networkx_edges(G_rbfs,pos_rbfs,edgelist=list(zip(path_rbfs,path_rbfs[1:])),edge_color='green',width=4,arrows=True)
nx.draw_networkx_labels(G_rbfs,pos_rbfs,labels={n:f"{n}\nh(n)={h_rbfs[n]}" for n in graph_rbfs},font_size=9,font_weight='bold')
nx.draw_networkx_edge_labels(G_rbfs,pos_rbfs,edge_labels={(u,v):f"{w} km" for u,v,w in G_rbfs.edges.data('weight')},font_color='red')
plt.title("RBFS Route\nAgra to Gorakhpur")
plt.axis('off')
plt.tight_layout()
plt.show()
```

Output:-

```
"IDLE Shell 3.13.5"
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

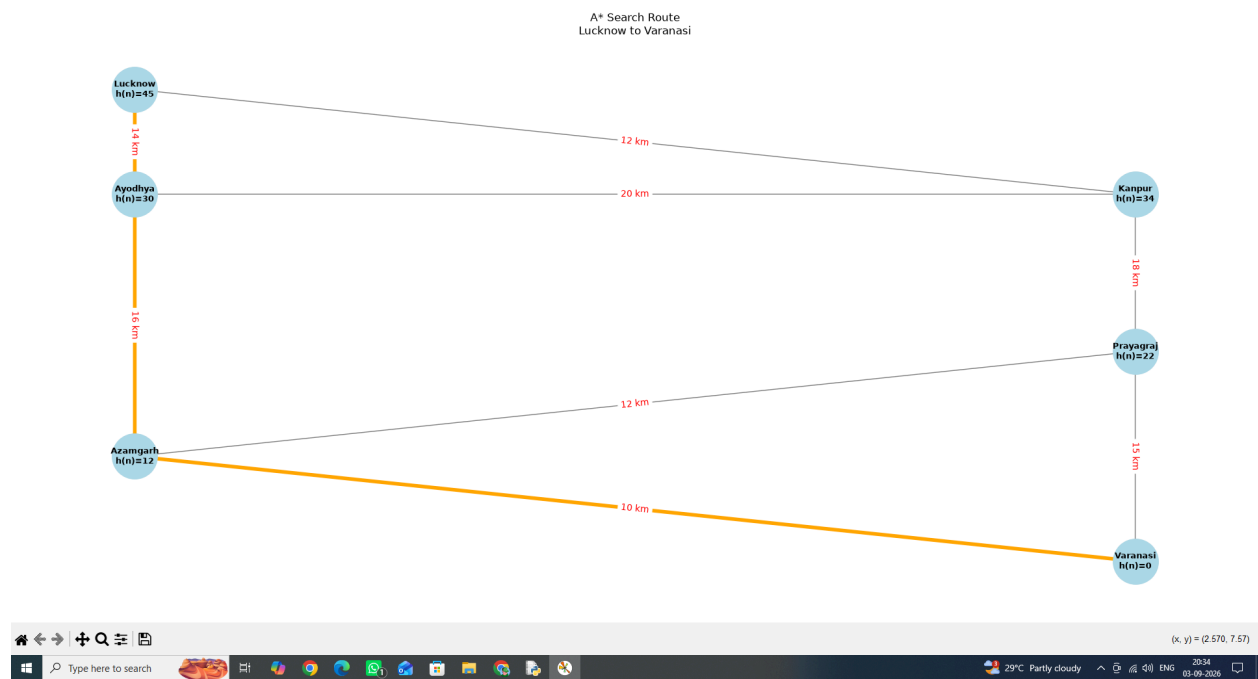
>>>
===== RESTART: D:\Hari\AI\Practical 2.py =====
Hariprasad Vishwakarma T127
=== A* Search Algorithm ===
Optimal Path Found:
Lucknow -> Ayodhya -> Azamgarh -> Varanasi
Total Distance = 40 km
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE

A.I. Practical No.9

Figure 1



```
"IDLE Shell 3.13.5"
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: D:\Hari\A.I\Practical 2.py =====
Hariprasad Vishwakarma T127
=== A* Search Algorithm ===
Optimal Path Found:
Lucknow -> Ayodhya -> Azamgarh -> Varanasi
Total Distance = 40 km

=== Recursive Best-First Search (RBFS) ===
Optimal Path Found:
Agra -> Mathura -> Kanpur -> Gorakhpur
Total Distance = 71 km
|
```

Hariprasad Vishwakarma
T127
TYCS

MVLU COLLEGE
A.I. Practical No.9

Figure 1

